

# 멈춘 작업을 다시 시작하고, 틀어진 데이터를 바로잡았습니다

Java·Kotlin·Spring으로 백엔드를 개발합니다. 문제가 생긴 이유와 해결한 과정을 네 가지 작업으로 정리했습니다.

서민재 · Rex Seo

Backend Engineer · Java / Kotlin / Spring  
ckdekrn88@gmail.com

GitHub

Velog

LinkedIn

이력서

경력기술서

01

## 이벤트가 실패한 단계를 남기고 필요한 업무만 다시 실행했습니다

|                                                                                                                                          |                                                           |                                                                                                                                    |                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| <b>담당</b><br>이벤트 상태·수동 복구·정체 알림 개발                                                                                                       | <b>구현</b><br>Kafka로 처리하는 내부 이벤트 9종 · 상태 6개 · 업무별 복구 방식 8개 | <b>저장</b><br>PostgreSQL 상태 기록 · JSONB payload                                                                                      | <b>알림</b><br>IN_PROGRESS 15분 초과 건을 타입별로 모아 Slack 전송 |
| <b>상황</b><br><b>‘실패’ 하나로 어느에서 다시 시작할지 알 수 없었습니다</b><br>발행 자체가 실패한 경우와 소비자가 처리하다 멈춘 경우에는 확인할 데이터와 복구할 업무가 달랐습니다. 하지만 둘 다 같은 실패 상태로 남았습니다. |                                                           | <b>바꾼 점</b><br><b>처리 단계와 복구 코드를 함께 나눴습니다</b><br>상태를 6개로 나눠 실패 단계를 기록했습니다. 포인트 차감·회수·만료 알림 등 8개 작업은 저장된 payload로 실패한 업무만 다시 실행했습니다. |                                                     |

## 배치는 중간부터 다시 시작하고, 동시 요청 뒤 잔액을 확인했습니다

소멸, 만료 알림, 아카이브 배치는 멈췄을 때 다시 시작할 위치가 각각 달랐습니다. 동시 요청 테스트에서는 최종 잔액과 원장, 이벤트 수를 함께 확인했습니다.

|                                                          |                                          |                                              |                                                    |
|----------------------------------------------------------|------------------------------------------|----------------------------------------------|----------------------------------------------------|
| <b>담당</b><br>원장 소멸 페이지 전환·<br>아카이브 배치 구현·만료 알림<br>이벤트 개선 | <b>처리 단위</b><br>전체 적재 대신 페이지 단위<br>반복 처리 | <b>보관 대상</b><br>참조 순서가 있는 archive<br>테이블 10개 | <b>동시성</b><br>9개 API · 13개 시나리오 ·<br>3·5·10개 동시 요청 |
|----------------------------------------------------------|------------------------------------------|----------------------------------------------|----------------------------------------------------|

### 상황

### 같은 원장을 다뤄도 다시 시작할 기준은 달랐습니다

소멸은 잔액과 원장을 바꾸고, 만료 안내는 알림을 쌓아 발송합니다. 아카이브는 참조 순서대로 테이블을 옮겨야 했습니다.

### 바꾼 점

### 각 배치에 이어서 처리할 위치를 남겼습니다

소멸·알림·아카이브는 진행 위치를 따로 기록했습니다. 포인트 거래와 예산 차감은 같은 락 키·동일 예산의 동시 요청이 끝난 뒤 잔액·원장·이벤트 수를 확인했습니다.

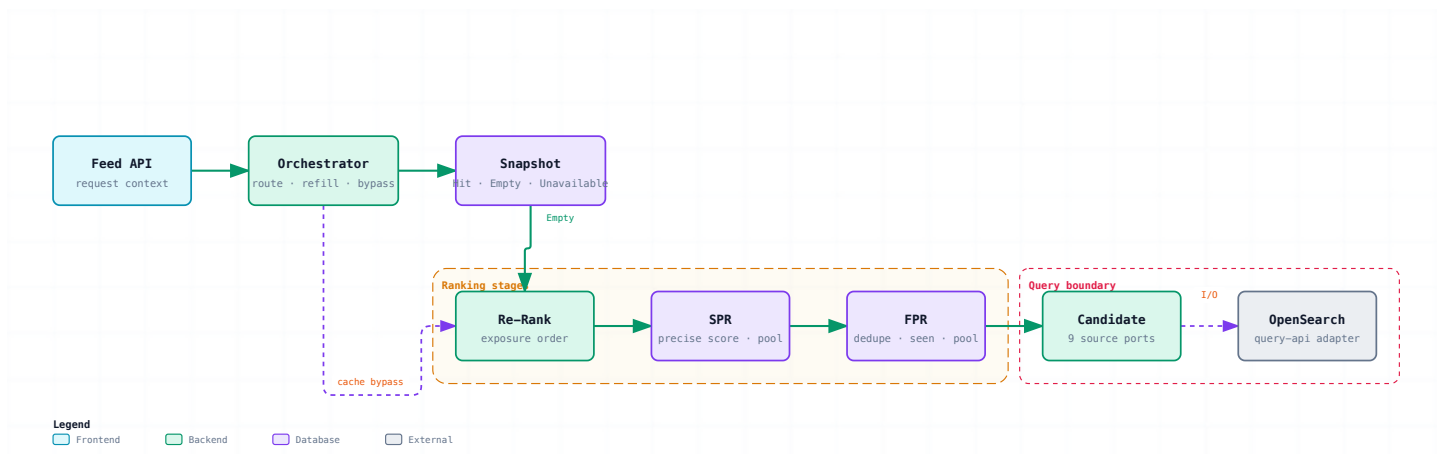
## 검색 소스별 실패 처리와 Redis 캐시 우회 경로를 구현했습니다

개인화 피드의 후보 조회, 랭킹, 캐시를 나누고 각 단계가 주고받는 값을 정리했습니다.

|                                                      |                                              |                                         |                                                        |
|------------------------------------------------------|----------------------------------------------|-----------------------------------------|--------------------------------------------------------|
| <b>처리 순서</b><br>후보 조회 → 중복 제거 → 점수화<br>→ 재정렬 → 캐시 저장 | <b>랭킹</b><br>후보 수집 인터페이스 · 역방향<br>조회 · 캐시 3종 | <b>담당</b><br>검색 소스 9개 정책·타임아웃·<br>실패 처리 | <b>연동</b><br>OpenSearch 조회 · Redis 캐시<br>우회 · 순환 의존 검출 |
|------------------------------------------------------|----------------------------------------------|-----------------------------------------|--------------------------------------------------------|

### 후보를 모아 피드로 내보내기까지

Snapshot 상태에 따라 캐시를 쓰거나 랭킹을 다시 실행합니다.



# 테스트마다 반복하던 준비 코드를 test-support로 모았습니다

Base, Mock, WebMvc, JPA 테스트의 시작점을 나눴습니다. fixture와 mock 준비, 느린 테스트 실행 여부를 공통 모듈에서 관리했습니다.

## 테스트 종류마다 필요한 준비만 사용합니다

공통 fixture부터 Mock, WebMvc, JPA 설정까지 단계별로 나눴습니다.

| 공통 기반                                                |  | Mock 테스트                                                  |  | MVC 테스트                                                              |  | JPA 테스트                                                                   |  |
|------------------------------------------------------|--|-----------------------------------------------------------|--|----------------------------------------------------------------------|--|---------------------------------------------------------------------------|--|
| 01 · fixture·생명주기                                    |  | 02 · 협력 객체 격리                                             |  | 03 · HTTP 요청·응답                                                      |  | 04 · 매핑·쿼리·영속성                                                            |  |
| BASE CLASS<br>AbstractTest<br>PER_CLASS · 공통 fixture |  | BASE CLASS<br>AbstractMockTest<br>mock 생성 · 초기화           |  | BASE CLASS<br>AbstractWebMvcTest<br>@WebMvcTest · Controller 자동 등록   |  | BASE CLASS<br>AbstractDataBaseTest<br>@DataJpaTest · Repository·Config 등록 |  |
| ↓                                                    |  | ↓                                                         |  | ↓                                                                    |  | ↓                                                                         |  |
| FIXTURE<br>FixtureMonkeyFactory<br>fixture 생성 규칙     |  | EXTENSION<br>AutoMockExtension<br>주입 · reset · session 정리 |  | WEB HELPER<br>WrappedMockMvc<br>요청 · 응답 검증용 래퍼                       |  | DATABASE HELPER<br>DB_INITIALIZER<br>Repository·Config Bean 등록            |  |
| 4개 테스트 진입점<br>Base · Mock · WebMvc · JPA             |  |                                                           |  | 5개 공통 도구·정책<br>Fixture · Response · MockMvc · Tx Support · Slow-test |  | 실행·작성 기준<br>skip.slow.test=true 제외 · 테스트 작성 원칙 제안                         |  |

## 공개한 코드와 글

기술 글 156편, YouthCon 발표, Spring 오픈소스에 반영된 개선 3건을 확인할 수 있습니다.

|                                                                                                       |                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>YouthCon 2025 발표</b><br>'우리팀을 위한 Spring Test 만들기'에서 테스트 코드를 줄이고 팀 규칙을 정한 과정을 발표했습니다.               | <b>기술 글 156편 · 2026.07 기준</b><br>Java·JPA·테스트·SQL 튜닝 과정에서 겪은 문제를 기록했습니다. 글도 10기 준비위원과 연사로도 참여했습니다.                                         |
| <b>Spring 오픈소스 · 개선 3건 반영</b><br>Kafka, Boot, Data JPA에 불변 컬렉션 처리, 자원 정리, 하드코딩 개선을 제안했고 프로젝트에 반영됐습니다. | <b>Java Cafe 기반 OpenSearch 한글 플러그인</b><br>Java Cafe Elasticsearch 플러그인을 OpenSearch 2.19·Kotlin으로 이식하고 한·영 자판 변환과 초성·자모 검색을 예시 16개로 확인했습니다. |
| <b>약 7만 건의 삭제 상태 불일치</b><br>파일 삭제 상태와 DB 기록이 어긋난 데이터 약 7만 건을 확인하고 원인을 분석한 과정을 썼습니다.                   | <b>JPA 오류 재현과 약 200만 건 보정</b><br>Mock 테스트가 놓친 JPA 변경 감지 오류를 재현하고 약 200만 건을 보정한 과정을 썼습니다.                                                   |